Task Manager API with AI Integration

Duration: 2 Hours
Total Marks: 100
Assessment Type: Individual Practical Assessment

🎯 Objective

Build a Task Manager REST API using Node.js, Express.js, MongoDB and Mongoose. Integrate an external AI API to generate intelligent task plans from user-provided descriptions.

The goal is to assess your backend development skills and your ability to research, understand and integrate a new API using official documentation and AI tools.

🛠️ Required Technologies

- Node.js
- Express.js
- MongoDB Atlas
- Mongoose
- dotenv
- External AI API (Groq API, Hugging Face or another accessible provider)
- Postman
- Render or another supported hosting platform

📌 Part 1: Task CRUD Operations — 35 Marks

Create a Task model with the following fields:

- `title` — required string
- `description` — string
- `status` — `pending`, `in-progress` or `completed`
- `dueDate` — optional date

Required Endpoints

Method	Endpoint	Description
POST	`/api/tasks`	Create a new task
GET	`/api/tasks`	Retrieve all tasks
GET	`/api/tasks/:id`	Retrieve a single task
PUT	`/api/tasks/:id`	Update a task
DELETE	`/api/tasks/:id`	Delete a task

Requirements

- Connect your application to MongoDB using Mongoose.
- Create a proper Mongoose schema and model.
- Implement all five CRUD operations.
- Validate required fields and task status.
- Handle invalid MongoDB IDs and tasks that do not exist.
- Return appropriate HTTP status codes and meaningful error messages.
- Keep your code organized into suitable routes, controllers and database/model files.

🤖 Part 2: AI Task Assistant — 35 Marks

Integrate an external AI API into your backend. You must research the provider's documentation and implement the integration yourself with the help of available resources.

Required Endpoint

```
POST /api/ai/plan-task
```

Example Request

```
{
  "description": "I need to build a portfolio website within three days."
}
```

Expected Response Format

```
{
  "success": true,
  "plan": {
    "summary": "Build and launch a personal portfolio website.",
    "priority": "high",
    "estimatedHours": 12,
    "steps": [
      "Plan the website structure",
      "Build the responsive layout",
      "Add projects and contact information",
      "Test and deploy the website"
    ]
  }
}
```

This is an example response. The actual plan will depend on the AI model.

Requirements

1. Research and select an accessible AI API.
2. Obtain an API key if required.
3. Store the API key in a `.env` file.
4. Accept a task description through the request body.
5. Send the description to the AI model from your backend.
6. Ask the AI to generate:
    - A short task summary
    - Priority (`low` , `medium` or `high`)
    - Estimated completion time in hours
    - A list of actionable steps
7. Return the generated result in a structured JSON response.
8. Handle empty descriptions, API failures and invalid AI responses.
9. Never expose your API key in frontend code or upload it to GitHub.

Important: You are not required to train an AI model. Your task is to integrate an existing AI API into your Node.js application.

Suggested Resources

- Groq Console: https://console.groq.com/
- Groq API Documentation: https://console.groq.com/docs/api-reference
- Hugging Face Inference Providers: https://huggingface.co/docs/inference-providers

You may use another suitable AI provider. Check its current free-tier availability and model requirements before implementation.

🧪 Part 3: Postman Testing and Error Handling — 15 Marks

Test your application using Postman.

Demonstrate the following:

- Creating a task successfully.
- Retrieving all tasks.
- Retrieving a single task by ID.
- Updating a task.
- Deleting a task.
- Handling an invalid task ID.
- Handling missing required fields.
- Generating an AI task plan.
- Handling a missing description or AI API failure.

Submit your Postman screenshots or collection.

🌐 Part 4: Deployment — 5 Marks

Attempt to deploy your backend to a supported hosting platform.

Requirements:

- Configure environment variables securely.
- Ensure the deployed application connects to MongoDB.
- Test the live CRUD endpoints.
- Test the AI endpoint if your provider's quota and deployment configuration allow it.

Submit your live API URL if deployment succeeds.

📦 Submission Requirements

Submit the following:

1. Source code or GitHub repository link.
2. Postman collection or testing screenshots.
3. Live API URL, if successfully deployed.
4. `.env.example` containing variable names only, without actual secrets.
5. A short explanation of how you integrated the AI API.

⏱️ Time Management

Time	Target
0–10 minutes	Read requirements and research an AI API
10–45 minutes	Implement MongoDB connection and CRUD
45–85 minutes	Research and integrate the AI API
85–105 minutes	Test endpoints and handle errors
105–120 minutes	Deployment attempt and code demonstration

Prioritize working CRUD operations and a successful AI integration before attempting deployment.

Final Goal: Build a working Task Manager REST API with complete CRUD operations and an external AI-powered task planning endpoint within two hours.